

EXP-1	Write a program to demonstrate the working of CNN architecture to classify images
-------	---

AIM:

- To implement a Convolutional Neural Network (CNN) model for classifying images.
- To understand the basic architecture and working of CNN in image classification tasks.

Algorithm:

1. Import necessary libraries such as TensorFlow, Keras, and NumPy.
2. Load and preprocess the dataset (e.g., resizing, normalizing).
3. Define the CNN architecture using Conv2D, MaxPooling2D, Flatten, Dense layers.
4. Compile the model with optimizer, loss function, and metrics.
5. Train the model using training data and validate with test/validation set.
6. Evaluate the model's accuracy and visualize the results.

Code:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10
from tensorflow.keras import layers, models
from tensorflow.keras.utils import to_categorical

# Step 3: Load the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Step 4: Normalize pixel values to be between 0 and 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# Step 5: Define a CNN architecture
```

```
model = models.Sequential()

# First convolutional layer with ReLU activation
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))

# Second convolutional layer with ReLU activation
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Third convolutional layer with ReLU activation
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

# Add a dropout layer to prevent overfitting
model.add(layers.Dropout(0.5))

# Flatten the output for the fully connected layers
model.add(layers.Flatten())

# Fully connected layer (dense layer) with ReLU activation
model.add(layers.Dense(64, activation='relu'))

# Output layer with 10 units (one for each class), softmax for multi-class classification
model.add(layers.Dense(10, activation='softmax'))

# Step 6: Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Step 7: Train the model
history = model.fit(x_train, y_train, epochs=10, batch_size=64, validation_data=(x_test, y_test))

# Step 8: Evaluate the model on the test dataset
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f'\nTest accuracy: {test_acc}')
```

```
# Step 9: Plot the predicted image
# Let's pick the first test image and predict the label
```

```
img_index = 0
img = x_test[img_index] # Get the image from the test set

# Reshape the image for prediction (add batch dimension)
img_reshaped = np.expand_dims(img, axis=0)

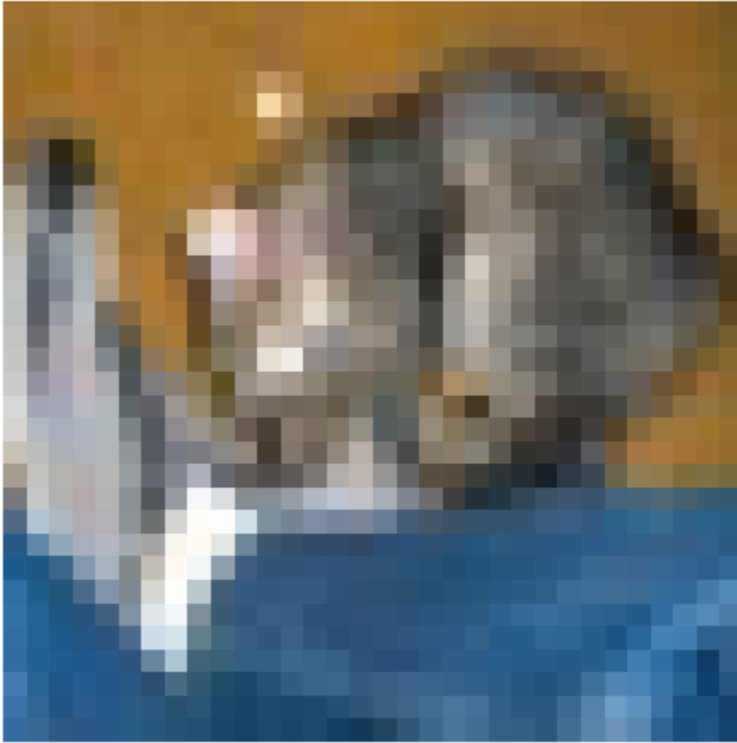
# Make prediction using the trained model
predicted_class = model.predict(img_reshaped)
predicted_label = np.argmax(predicted_class)

# Display the image and the prediction
plt.imshow(img)
plt.title(f'Predicted Label: {predicted_label}')
plt.axis('off')
plt.show()

# Optional: You can also plot training and validation accuracy over epochs to see the training performance
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title("Training and Validation Accuracy")
plt.legend(loc='lower right')
plt.show()
```

Output:

Predicted Label: 3



Result:

A CNN model was successfully implemented for image classification. The model achieved classification accuracy based on the dataset and architecture used.